Lab Exercise 18- Volume in Kubernetes Cluster

Name: Arin Agrawal

SAP ID:500120660

Roll no: R2142230502

Batch: Devops B1

Volumes in Kubernetes provide a way to store and share data among containers in a Pod. This lab exercise will guide you through the creation and use of different types of volumes in Kubernetes.

Step 1: Set Up Kubernetes Cluster

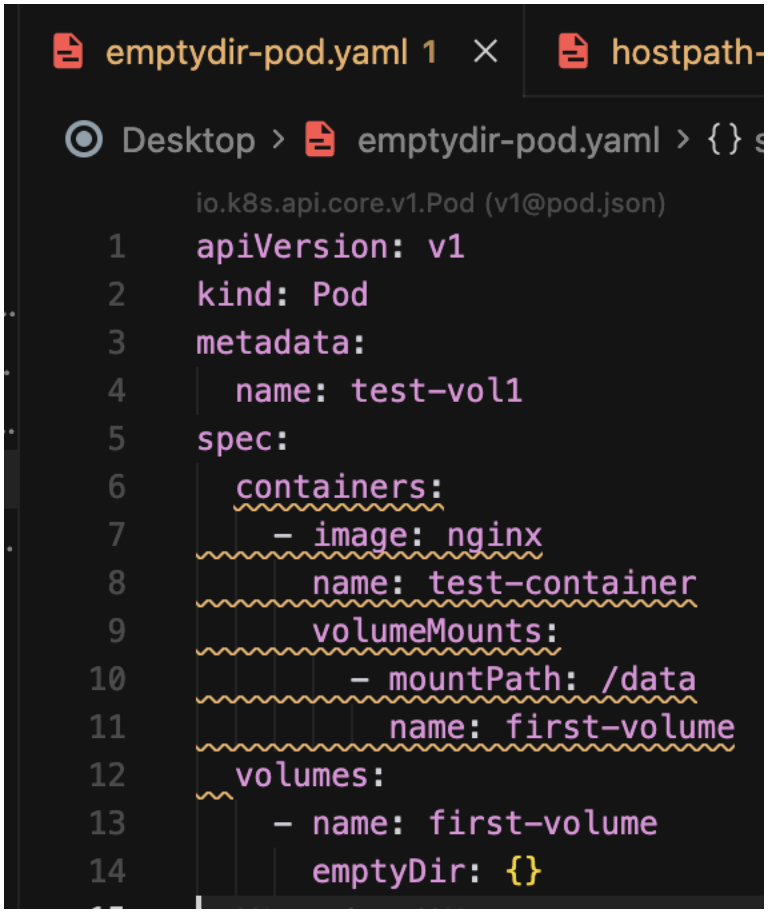
Ensure you have access to a Kubernetes cluster. You can use a local setup with Minikube, kind, or use a cloud-based Kubernetes service.

Step 2: Create a Pod with an emptyDir Volume

Create a file named **emptydir-pod.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-vol1
spec:
  containers:
    - image: nginx
      name: test-container
      volumeMounts:
        - mountPath: /data
```

```
    name: first-volume
volumes:
  - name: first-volume
    emptyDir: {}
```



In this manifest:

The emptyDir volume is a temporary storage that is created when the Pod is assigned to a node and exists as long as the Pod is running.

Apply the manifest to create the Pod:

```
kubectl apply -f emptydir-pod.yaml
```

```
AKSHs-MacBook-Air:Desktop akshchauhan$ kubectl apply -f emptydir-pod.yaml
pod/test-vol1 created
```

Step 3: Verify the Pod and Volume

Check the status of the Pod:

```
kubectl get pods
```

```
AKSHs-MacBook-Air:Desktop akshchauhan$ kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
test-vol1   1/1     Running   0          6s
```

Access the Pod's shell:

```
kubectl exec -it test-vol1 -- bash
```

Inside the Pod, create a file in the /data directory:

```
touch f1 f2 f3
```

Exit the Pod shell:

```
exit
```

```
root@test-vol1:/# cd data
root@test-vol1:/data# touch f1 f2 f3
root@test-vol1:/data# ls
f1  f2  f3
root@test-vol1:/data# exit
exit
```

Step 4: Create a Pod with a hostPath Volume

Create a file named **hostpath-pod.yaml** with the following content:

```
apiVersion: v1
```

```yaml
kind: Pod
metadata:
  name: test-vol2
spec:
  containers:
    - image: coolgourav147/nginx-custom
      name: test-container
      volumeMounts:
        - mountPath: /data
          name: first-volume
  volumes:
    - name: first-volume
      hostPath:
        path: /test
```

emptydir-pod.yaml 1 | hostpath-pod.yaml 1

Desktop > hostpath-pod.yaml > {} spec > [ ] volume

io.k8s.api.core.v1.Pod (v1@pod.json)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-vol2
spec:
  containers:
  - name: test-container
    image: nginx
    volumeMounts:
    - mountPath: /data
      name: first-volume
  volumes:
  - name: first-volume
    hostPath:
      path: /test
      type: DirectoryOrCreate
```

In this manifest:

The hostPath volume mounts a directory from the host into the Pod at /mnt/data.

The busybox container writes the current date to a log file every 5 seconds.

Apply the manifest to create the Pod:

```
kubectl apply -f hostpath-pod.yaml
```

```
pod/test-vol2 created
AKSHs-MacBook-Air:Desktop akshchauhan$
```

Step 3: Verify the Pod and Volume

Check the status of the Pod:

```
kubectl get pods
```

```
NAME        READY   STATUS    RESTARTS   AGE
test-vol1   1/1     Running   0          6m56s
test-vol2   1/1     Running   0          9s
```

Access the Pod's logs to see the data being written:

```
kubectl exec -it test-vol2 -- bash
```

```
root@test-vol2:/# cd data
root@test-vol2:/data# ls
file1
root@test-vol2:/data# cat file1
root@test-vol2:/data# exit
exit
```

You should see the dates being appended to the log file.

Check the host directory to verify the volume:

If you have access to the node running the Pod, you can verify the data in /tmp/data/log.txt.

```
Linux minikube 6.12.69-linuxkit #1 SMP Mon Feb 16 11:19:06 UTC 2026 aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
docker@minikube:~$ ls /test
file1
```